Hospital Surgery Scheduling System

Using Linked List and Directed Acyclic Graph

C Data Structures Project— IEEE Format Report

Team Members

Member 1: V.Charani

Member 2: G.L.Sai Teja

Table of contents

1. Abstract
2. Introduction
3. Objectives
4. Scope of the Project
5. Literature Review
6. Methodology
7. Technology Used
8. Architecture / System Design
9. Implementation Details
10. Results / Output
11. Conclusion
12. Future Work
13. Footnotes / References

ABSTRACT

This project presents a Hospital Surgery Scheduling System implemented using the C programming language. The system manages surgical procedures using a linked list for CRUD operations and a Directed Acyclic Graph (DAG) for handling dependencies between surgeries. The application allows users to add, update, delete, search, and display surgeries while also defining prerequisite relationships. A topological sorting algorithm is used to determine the optimal sequence of surgeries. This system demonstrates efficient data handling and scheduling in healthcare environments.

INTRODUCTION:

Efficient scheduling in hospitals is critical to ensure optimal utilization of operating rooms and medical staff. Surgeries often depend on prior procedures, making scheduling complex.

This project solves the problem using:

- Linked List ➔ for dynamic CRUD operations
- Directed Acyclic Graph (DAG) ➔ for dependency management
- Topological Sorting ➔ for scheduling order

The system is menu-driven and user-friendly.

OBJECTIVES:

- To implement CRUD operations using linked lists
- To model surgery dependencies using DAG
- To generate a valid surgery schedule using topological sort
- To demonstrate modular programming in C
- To build a real-world healthcare application

SCOPE OF THE PROJECT:

The system can be used in:

- Hospitals for scheduling surgeries
- Clinics managing operation workflows
- Educational purposes (data structures learning)

Limitations:

- Console-based (no GUI)
- Limited to small datasets (MAX = 50)

LITERATURE REVIEW:

Previous research in scheduling systems shows:

- Graph-based models are effective for dependency handling
- Topological sorting is widely used in scheduling tasks
- Linked lists provide flexible memory management

This project combines these concepts into a practical system.

METHODOLOGY:

The system follows these steps:

1. Input surgeries
2. Store using linked list
3. Define dependencies using adjacency matrix
4. Perform CRUD operations
5. Apply DFS-based Topological Sort
6. Display scheduling order

TECHNOLOGY USED:

- Programming Language: **C**
- Concepts:
  - Data Structures (Linked List, Graph)
  - Dynamic Memory Allocation (malloc, free)
  - DFS Algorithm
- Compiler: GCC

ARCHITECTURE/SYSTEM DESIGN:

Components:

- Linked List Module
  - Stores surgery details (ID, Name, Duration)
- Graph Module

    - Adjacency matrix for dependencies
- Scheduler Module
    - DFS + Stack for topological sorting
- Menu Module
    - User interaction

Workflow:

User Input ➔ CRUD Operations ➔ Dependency Graph ➔ Topological Sort ➔Output

IMPLEMENTATION DETAILS:

Data Structure:

```
typedef struct Surgery {
    int id;
    char name[50];
    int duration;
    struct Surgery* next;
} Surgery;
```

Features Implemented:

- Create (Add Surgery)
- Read (Display)
- Update
- Delete
- Search
- Add Dependency
- Display Graph
- Topological Sort

Algorithm:

- DFS (Depth First Search) used for topological sorting
- Stack used to store order

RESULTS/OUTPUT:

SampleOutput:

```
===== HOSPITAL SCHEDULING MENU =====
1. Add Surgery
2. Delete Surgery
3. Update Surgery
4. Search Surgery
5. Display Surgeries
6. Add Dependency (Edge)
7. Show Graph
8. Get Schedule (Topological Sort)
9. Exit
Enter choice:
```

Example:

```
Surgery added!
Dependency added: 1 -> 2

--- Surgery Schedule Order ---
1 2 3
```

CONCLUSION:

The project successfully demonstrates how data structures like linked lists and graphs can be applied to solve real-world problems such as hospital scheduling. The use of topological sorting ensures that all dependencies are respected, resulting in an efficient and valid surgery schedule.

FUTURE WORK:

- Add GUI interface
- Use database instead of memory
- Handle large-scale data
- Add priority-based scheduling
- Integrate real-time hospital systems

FOOTNOTES/REFERENCES:

- Data Structures using C – Reema Thareja
- Introduction to Algorithms
- GeeksforGeeks – Topological Sorting
- IEEE Documentation Guidelines
- Used google for reference GitHub ieee format

https://github.com/Vatshayan/IEEE-Projects/blob/main/Shivam Project Synopsis.pdf